

```
int& getValue() {
    static int a = 10;
    return a;
}
int main() {
    getValue() = 22; // modify through function
    cout << getValue(); // 22
}
```

```
const int& getValue() {
    static int a = 10;
    return a;
}
int main() {
    // getValue() = 22; read-only
    cout << getValue(); // 10
}
```

```
int a = 10;
int& getValue() {
    return a;
}
int main() {
    getValue() = 50; // modify through function
    cout << a; // 50
}
```

```
int a = 10;
const int& getValue() {
    return a;
}
int main() {
    // getValue() = 50; read-only
    cout << getValue(); // 10
}
```

```
int& getValue(int& x) {
    return x;
}
int main() {
    int a = 10;
    getValue(a) = 99; // modify through function
    // cout << getValue(20); // 20
    cout << a; // 99
}
```

```
const int& getValue(const int& x) {
    return x;
}
int main() {
    int a = 10;
    // getValue(a) = 99; read-only
    cout << getValue(a); // 10
    cout << getValue(20); // 20
}
```

```
const int& func() {
    const int x = 10; // local variable
    return x; // ❌ DANGEROUS
}
const int& r = func(); // ❌ r is dangling
```

```
const int& func() {
    return 20; // temporary
}
const int& r = func(); // ❌ still unsafe!
```

const int& z = 20; // ✅ valid A const reference can bind to both lvalues and rvalues

- 👉 const prevents modification
- 👉 which makes binding to a temporary safe
- 👉 and then C++ allows lifetime extension

```
const int w = 10; // ✅ valid
const int& x = w;
int& y = 20; // ❌ not allowed
```

```
int a=10; int &b=a; cout<<a<<b; /* b=20; */ a=20; cout<<a<<b; // 10102020
```

This is a classic C++ pointer concept

const int* p;

✓ You CANNOT change value

✓ You CAN change pointer

int a = 10, b = 20;

// pointer → const int

p = &a; // ✓ OK

p = &b; // ✓ OK

***p = 30;** // ✗ ERROR (value is const)

int* const p;

✓ You CAN change value

✓ You CANNOT change pointer

int a = 10, b = 20;

// const pointer → int

p = &a

***p = 30;** // ✓ OK

p = &b; // ✗ ERROR (pointer is const)

const int* const p = &a;

✓ You CANNOT change value

✓ You CANNOT change pointer

// const pointer → const int

***p = 30;** // ✗ ERROR

p = &b; // ✗ ERROR

This is a classic C++ reference concept

```
int& ref = a;
```

```
int a = 10, b = 20;
```

```
ref = 30; // ✓ a becomes 30
```

```
ref = b; // ✓ a becomes 20 (copies value)
```

```
int& r = 10; // ✗ ERROR (temporary)
```

```
// lvalue reference → int
```

✓ Refers to an existing object (lvalue)

✓ You CAN change value

✓ You CANNOT rebind reference

```
const int& ref = a;
```

```
int a = 10;
```

```
const int& r1 = a; // ✓ OK
```

```
const int& r2 = 10; // ✓ OK
```

```
r1 = 20; // ✗ ERROR
```

```
// const lvalue reference → const int
```

✓ Refers to lvalue

✓ Can also bind to temporary (rvalue)

✓ You CANNOT change value through reference

```
int&& ref = 10;
```

```
int a = 10;
```

```
int&& r1 = 20; // ✓ OK
```

```
int&& r2 = a; // ✗ ERROR
```

```
r1 = 30; // ✓ OK
```

```
// rvalue reference → int
```

✓ Refers to temporary object (rvalue)

✓ You CAN change value

✓ Cannot bind directly to lvalue

```
const int&& ref = 10;
```

```
const int&& r = 10; // ✓ OK
```

```
r = 20; // ✗ ERROR
```

```
// const rvalue reference → const int
```

✓ Refers to temporary object (rvalue)

✓ You CANNOT change value
